

✓ Inferencing with Bayesian Networks in Python

Lung Cancer Diagnosis using Bayesian Network

Aim

To construct a Bayesian Network for Lung Cancer diagnosis and perform probabilistic inference using Python.

Objective

- Build a Bayesian Network using pgmpy.
 - Define Conditional Probability Distributions (CPDs).
 - Verify the model.
 - Perform probabilistic inference.
 - Predict the probability of lung cancer under different conditions.
-

Problem Statement

A person's risk of developing lung cancer depends primarily on two factors:

- **Pollution** (Low / High)
- **Smoking** (Yes / No)

If a person develops **Cancer**, it influences:

- **X-ray Result** (Positive / Negative)
- **Dyspnoea (Breathlessness)** (Yes / No)

The Bayesian Network models these probabilistic relationships and allows us to answer diagnostic and predictive queries.

Step 1: Install the Required Library (pgmpy)

pgmpy (Probabilistic Graphical Models using Python) is an open-source library

Required to :

- Create a Bayesian Network structure.
- Define Conditional Probability Distributions (CPDs).
- Validate whether the network is correctly built.
- Perform probabilistic inference (answer probability queries).

- Learn Bayesian Networks from data

```
1 # Install pgmpy (Run only once in Google Colab)
2
3 !pip install pgmpy
```

Collecting pgmpy

Downloading pgmpy-1.1.2-py3-none-any.whl.metadata (13 kB)

Requirement already satisfied: huggingface_hub>=0.23 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (0.23.0)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (3.0)
Requirement already satisfied: numpy>=2.0 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (2.0.2)
Requirement already satisfied: scipy>=1.10 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (1.10.1)
Requirement already satisfied: scikit-learn>=1.2 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (1.2.2)
Requirement already satisfied: pandas>=1.5 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (2.0.3)
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (0.14.5)
Requirement already satisfied: tqdm>=4.64 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (4.64.1)
Requirement already satisfied: pyparsing>=3.0 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (3.0.9)
Requirement already satisfied: joblib>=1.2 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (1.2.0)
Requirement already satisfied: opt_einsum>=3.3 in /usr/local/lib/python3.12/dist-packages (from pgmpy) (3.3.0)
Collecting scikit-base>=0.12.4 (from pgmpy)

Downloading scikit_base-1.0.2-py3-none-any.whl.metadata (8.8 kB)

Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (8.1.7)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (3.10.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (2023.5.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (0.23.0)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (20.9)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (6.0.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (4.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (2020.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (2022.7)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (3.1.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (0.5.6)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from scikit-base) (3.5.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from scikit-base) (2023.11.17)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from scikit-base) (1.0.2)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from scikit-base) (3.4)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (0.14.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from scikit-base) (1.16.0)
Downloading pgmpy-1.1.2-py3-none-any.whl (2.4 MB)

2.4/2.4 MB 38.7 MB/s eta 0:00:00

Downloading scikit_base-1.0.2-py3-none-any.whl (165 kB)

165.3/165.3 kB 10.1 MB/s eta 0:00:00

Installing collected packages: scikit-base, pgmpy

Successfully installed pgmpy-1.1.2 scikit-base-1.0.2

Explanation

- ! tells Google Colab to execute the command in the system terminal.
- pip is Python's package manager.
- install downloads and installs the specified package.

- pgmpy is the library being installed.

Purpose of Each Library

✓ Library Purpose

BayesianNetwork Creates the structure of the Bayesian Network (nodes and edges).

- TabularCPD Defines the probability tables (CPDs) for each variable.
- VariableElimination Performs inference and answers probability queries.
- NetworkX Creates and manages the network graph.
- Matplotlib Displays the Bayesian Network graph visually.

```
1 # Import the Bayesian Network model
2 try:
3     # For newer versions of pgmpy
4     from pgmpy.models import DiscreteBayesianNetwork as BayesianNetwork
5 except ImportError:
6     # For older versions of pgmpy
7     from pgmpy.models import BayesianNetwork
8
9 # Import Conditional Probability Distribution (CPD)
10 from pgmpy.factors.discrete import TabularCPD
11
12 # Import the inference algorithm
13 from pgmpy.inference import VariableElimination
14
15 # Import libraries for visualization
16 import networkx as nx
17 import matplotlib.pyplot as plt
```

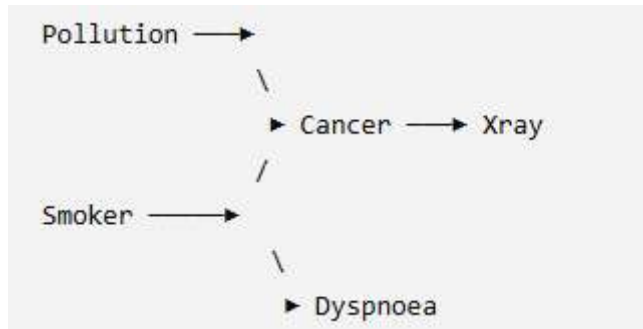
```
/usr/local/lib/python3.12/dist-packages/pgmpy/estimators/__init__.py:4: FutureWarning
from .StructureScore import (
```

Step 3: Create the Bayesian Network Structure

In this step, we define the structure of the Bayesian Network by specifying the relationships (edges) between the variables.

```
1 # Create the Bayesian Network
2
3 model = BayesianNetwork([
4     ('Pollution', 'Cancer'),
5     ('Smoker', 'Cancer'),
6     ('Cancer', 'Xray'),
```

```
7     ('Cancer', 'Dyspnoea')
8 ])
```



Step 4: Define the Conditional Probability Distributions (CPDs)

```
1 cpd_pollution = TabularCPD(  
2     variable='Pollution',  
3     variable_card=2,  
4     values=[[0.90], [0.10]]  
5 )
```

Explanation

Pollution is a root node (it has no parent), so we assign its prior probabilities directly.

This means:

- There is a 90% chance of Low Pollution.
- There is a 10% chance of High Pollution.

```
1 cpd_smoker = TabularCPD(  
2     variable='Smoker',  
3     variable_card=2,  
4     values=[[0.70], [0.30]]  
5 )
```

Explanation

Smoker is also a root node, so we assign prior probabilities.

This means:

- 70% of people are Non-Smokers.
- 30% of people are Smokers.

```
1 cpd_cancer = TabularCPD(  
2     variable='Cancer',
```

```

3     variable_card=2,
4     values=[
5         [0.97, 0.90, 0.95, 0.75], # No Cancer
6         [0.03, 0.10, 0.05, 0.25] # Cancer
7     ],
8     evidence=['Pollution', 'Smoker'],
9     evidence_card=[2, 2]
10 )

```

Explanation

The probability of Cancer depends on Pollution and Smoking.

Pollution	Smoker	No Cancer	Cancer
Low	No	0.97	0.03
Low	Yes	0.90	0.10
High	No	0.95	0.05
High	Yes	0.75	0.25

Observation: The highest probability of cancer (25%) occurs when a person is exposed to High Pollution and Smokes.

```

1 cpd_xray = TabularCPD(
2     variable='Xray',
3     variable_card=2,
4     values=[
5         [0.80, 0.10], # Negative
6         [0.20, 0.90] # Positive
7     ],
8     evidence=['Cancer'],
9     evidence_card=[2]
10 )

```

Explanation

The X-ray result depends only on whether the person has cancer.

Cancer	Negative X-ray	Positive X-ray
No	0.80	0.20
Yes	0.10	0.90

Observation:

- Cancer increases the probability of a Positive X-ray.
- A healthy person is more likely to have a Negative X-ray.

```

1 cpd_dyspnoea = TabularCPD(
2     variable='Dyspnoea',
3     variable_card=2,
4     values=[
5         [0.70, 0.35], # No Dyspnoea
6         [0.30, 0.65] # Dyspnoea
7     ],
8     evidence=['Cancer'],
9     evidence_card=[2]
10 )

```

Explanation

This CPD defines the probability of Dyspnoea (breathlessness) based on whether the person has Cancer.

Cancer	No Dyspnoea	Dyspnoea
No (0)	0.70	0.30
Yes (1)	0.35	0.65

Summary

Pollution and Smoker are root nodes, so they use prior probabilities.

Cancer depends on Pollution and Smoker.

Xray depends on Cancer.

Dyspnoea depends on Cancer.

Key Point

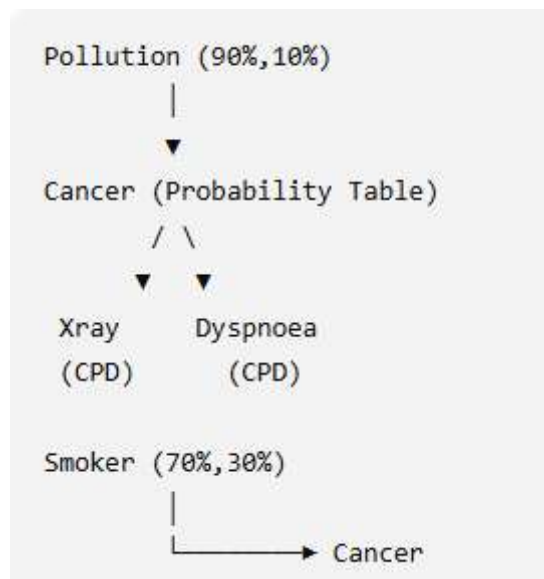
In this step, we assign probabilities to every variable in the network. These probability tables (CPDs) allow the Bayesian Network to calculate the likelihood of different outcomes during inference.

Step 6: Add the CPDs to the Bayesian Network

```
1 # Add all CPDs to the Bayesian Network
2 model.add_cpds(
3     cpd_pollution,
4     cpd_smoker,
5     cpd_cancer,
6     cpd_xray,
7     cpd_dyspnoea
8 )
```

Explanation

The `add_cpds()` function attaches each CPD to its corresponding node in the Bayesian Network.



Step 7: Validate the Bayesian Network

The `check_model()` function checks whether:

- Every node has a CPD.
- The probabilities in each CPD sum to 1.
- The CPDs match the network structure.
- The Bayesian Network is valid.

```
1 # Check whether the Bayesian Network is valid
2 print(model.check_model())
```

True

Explanation

The `check_model()` function validates the entire Bayesian Network.

Step 8: Create the Inference Engine

```
1 # Import the inference algorithm
2 from pgmpy.inference import VariableElimination
3
4 # Create the inference engine
5 inference = VariableElimination(model)
```

Explanation:

Import VariableElimination - This imports the Variable Elimination algorithm, which is one of the most common methods used for probabilistic inference in Bayesian Networks. It calculates probabilities based on the network structure and CPDs.

Create the Inference Object - Here, `model` is our validated Bayesian Network. `VariableElimination(model)` creates an inference engine for that model. The inference engine is stored in the variable `inference`.

Step 9: Perform Inference

```
1 # Query 1: Probability of Cancer
2 # Evidence:
3 # Pollution = High (1)
4 # Smoker = Yes (1)
5
6 query1 = inference.query(
7     variables=['Cancer'],
8     evidence={
9         'Pollution': 1,
10        'Smoker': 1
11    }
12 )
13
14 print(query1)
```

```
+-----+-----+
| Cancer | phi(Cancer) |
```


Cancer	phi(Cancer)
Cancer(0)	0.7500
Cancer(1)	0.2500

Explanation: This query uses the Variable Elimination inference algorithm to calculate the probability of Cancer. It takes High Pollution (Pollution = 1) and Smoking (Smoker = 1) as known evidence and computes the likelihood of the patient having or not having cancer. The result displays the probability distribution for the Cancer variable based on the given conditions.

```

1 # Query 2: Probability of Cancer
2 # Evidence:
3 # Xray = Positive (1)
4
5 query2 = inference.query(
6     variables=['Cancer'],
7     evidence={
8         'Xray': 1
9     }
10 )
11
12 print(query2)

```

Cancer	phi(Cancer)
Cancer(0)	0.7865
Cancer(1)	0.2135

Explanation: This query calculates the probability of Cancer using the Variable Elimination inference algorithm. It considers a Positive X-ray (Xray = 1) as the known evidence and updates the probability of the patient having or not having cancer. The output displays the probability distribution for the Cancer variable based on the X-ray result.

```

1 # Query 3: Probability of Dyspnoea
2 # Evidence:
3 # Cancer = Yes (1)
4
5 query3 = inference.query(
6     variables=['Dyspnoea'],
7     evidence={
8         'Cancer': 1
9     }

```

```

10 )
11
12 print(query3)

```

```

+-----+-----+
| Dyspnoea | phi(Dyspnoea) |
+=====+=====+
| Dyspnoea(0) | 0.3500 |
+-----+-----+
| Dyspnoea(1) | 0.6500 |
+-----+-----+

```

Explanation: This query calculates the probability of Dyspnoea (breathlessness) using the Variable Elimination inference algorithm. It takes Cancer = Yes (Cancer = 1) as the known evidence and estimates the likelihood of the patient experiencing or not experiencing dyspnoea. The output displays the probability distribution for the Dyspnoea variable based on the presence of cancer.

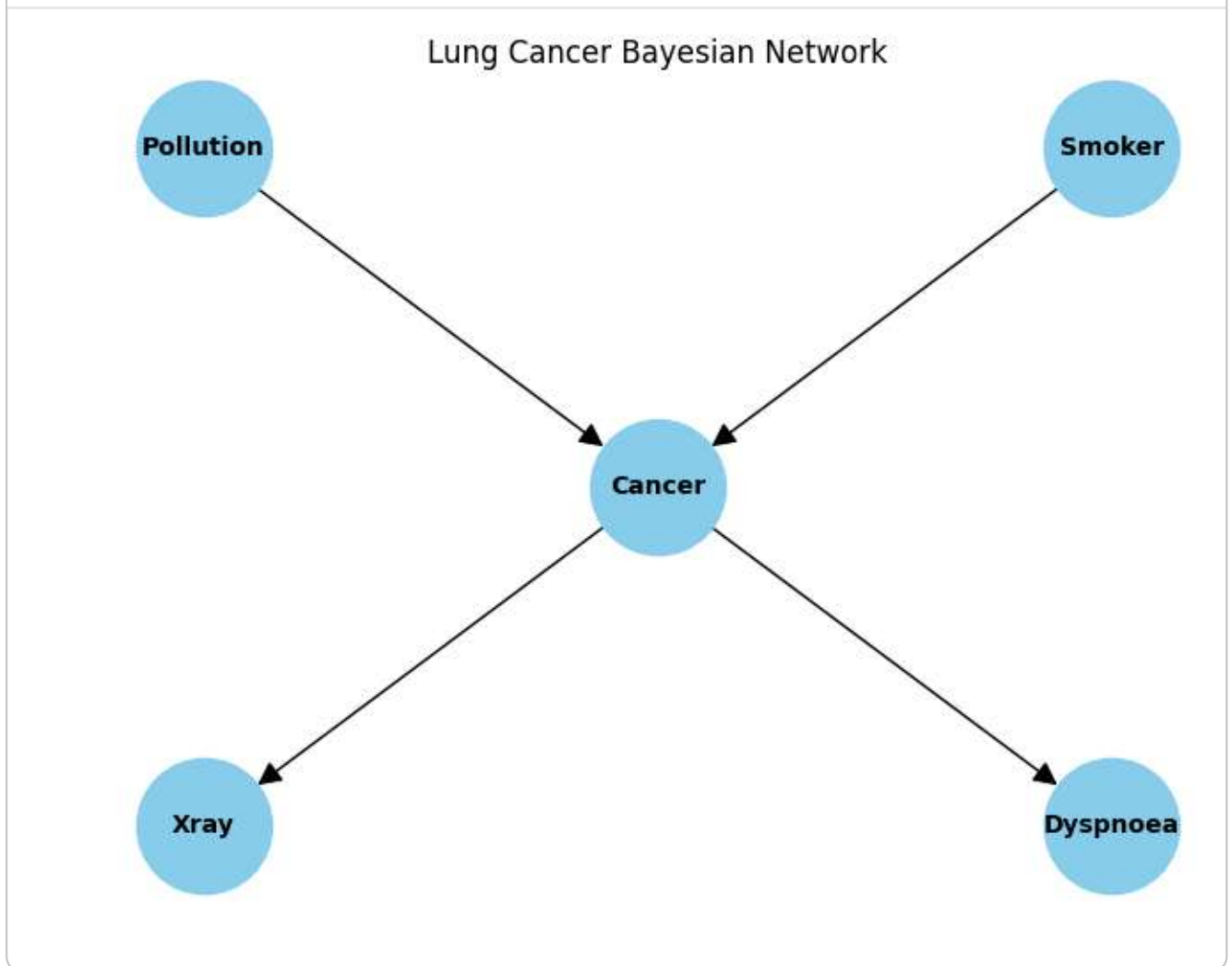
Step 12: Visualize the Bayesian Network

```

1 import matplotlib.pyplot as plt
2 import networkx as nx
3
4 G = nx.DiGraph(model.edges())
5
6 plt.figure(figsize=(8,6))
7
8 pos = {
9     "Pollution": (-1, 1),
10    "Smoker": (1, 1),
11    "Cancer": (0, 0),
12    "Xray": (-1, -1),
13    "Dyspnoea": (1, -1)
14 }
15
16 nx.draw_networkx(
17     G,
18     pos,
19     node_color="skyblue",
20     node_size=3000,
21     font_size=10,
22     font_weight="bold",
23     arrows=True,
24     arrowsize=20
25 )
26
27 plt.title("Lung Cancer Bayesian Network")

```

```
28 plt.axis("off")
29 plt.show()
```



Explanation: Visualizing the Bayesian Network helps understand relationships between variables, verify the network structure, and clearly identify parent and child nodes. It also improves the presentation and interpretation of the model. NetworkX is used to create the graph, while Matplotlib is used to visualize it. The directed graph shows the dependencies among all variables.

Conclusion

In this experiment, we successfully implemented a Bayesian Network for Lung Cancer Diagnosis using the pgmpy library in Python.

The network modeled the probabilistic relationships among five variables:

- Pollution
- Smoker
- Cancer
- Xray

- Dyspnoea

We defined the network structure and created Conditional Probability Distributions (CPDs) for each variable. After validating the model using `check_model()`, we used the Variable Elimination algorithm to perform probabilistic inference.

The Bayesian Network was able to answer diagnostic questions such as:

- What is the probability of cancer given high pollution and smoking?
- What is the probability of cancer given a positive X-ray?
- What is the probability of dyspnoea if the patient has cancer?

Finally, we visualized the Bayesian Network to understand the dependencies between variables.

This experiment demonstrates how Bayesian Networks can be used for medical diagnosis under uncertainty, helping estimate disease probabilities from observed risk factors, symptoms, and test results.

--MADE BY ASHWIN